XGBoost: Why Gradient Boosting Wins

A Hyperparameter Deep-Dive Using NBA Player Statistics

Author: Srikar Reddy
Student ID: 23075819
Dataset: NBA Players Stats 2023-24 Season (Basketball Reference)
GitHub:
https://github.com/Srikar9849598506/XGBoost-Why-Gradient-Boosting-Wins-23075819/tree/main

---

1. Introduction

XGBoost (Chen & Guestrin, 2016) has won more Kaggle competitions than any other algorithm. It dominates tabular data tasks — structured datasets with rows and columns — where deep learning often fails to offer an advantage. Yet many practitioners use it as a black box, tuning parameters by trial and error without understanding what they control.

This tutorial explains the mechanics of gradient boosting from first principles, then demonstrates through systematic experiments on **NBA 2023-24 player statistics** exactly how three core hyperparameters — **n_estimators**, **learning_rate**, and **max_depth** — interact to determine model quality. The task is predicting player position (Guard / Forward / Centre) from per-game statistics.

> • *Central insight: XGBoost is not magic — it is a principled combination of weak learners guided by gradient descent. Understanding the hyperparameters means understanding the algorithm.*

---

2. From Decision Trees to Gradient Boosting

2.1 The weakness of a single tree

A decision tree partitions the feature space into rectangular regions, assigning a prediction to each. It is interpretable and fast but suffers from **high variance** — small changes in training data produce very different trees. Accuracy on unseen data is limited because the model memorizes training data rather than generalizing.

2.2 Gradient boosting: sequential error correction

Friedman (2001) proposed building an ensemble of trees *sequentially*, where each new tree corrects the residual errors of all previous trees. The ensemble prediction is:

```
F_0(x) = initial prediction (class frequencies)
F_m(x) = F_{m-1}(x) + learning_rate * tree_m(x)
         where tree_m fits the residuals of F_{m-1}
```

Each tree is a "weak learner" — deliberately shallow and imprecise. The ensemble accumulates many small improvements rather than relying on one complex model. The **learning_rate** shrinks each tree's contribution, preventing any single tree from dominating.

3. Dataset: NBA 2023-24 Player Statistics

We use per-game statistics from the 2023-24 NBA season (Basketball Reference). The dataset contains 75 players with 13 features: games played, minutes, points, total rebounds, assists, steals, blocks, turnovers, field goal %, 3-point %, free throw %, offensive rebounds, and personal fouls.

The prediction target is position group: **Centre**, **Forward**, or **Guard**. This is a genuinely hard classification problem because modern NBA players are positionally versatile — many forwards shoot three-pointers like guards, while some guards rebound like forwards. This overlap makes the dataset ideal for demonstrating where a single decision tree fails but XGBoost succeeds.

Feature	Description	Why it predicts position
BLK	Blocks per game	Centres block far more than guards
TRB	Total rebounds per game	Interior players dominate rebounding
AST	Assists per game	Guards average more assists
P3_PCT	3-point percentage	Guards and forwards shoot more 3s
ORB	Offensive rebounds	Strongest centre indicator
PTS	Points per game	Less positional signal; all positions score

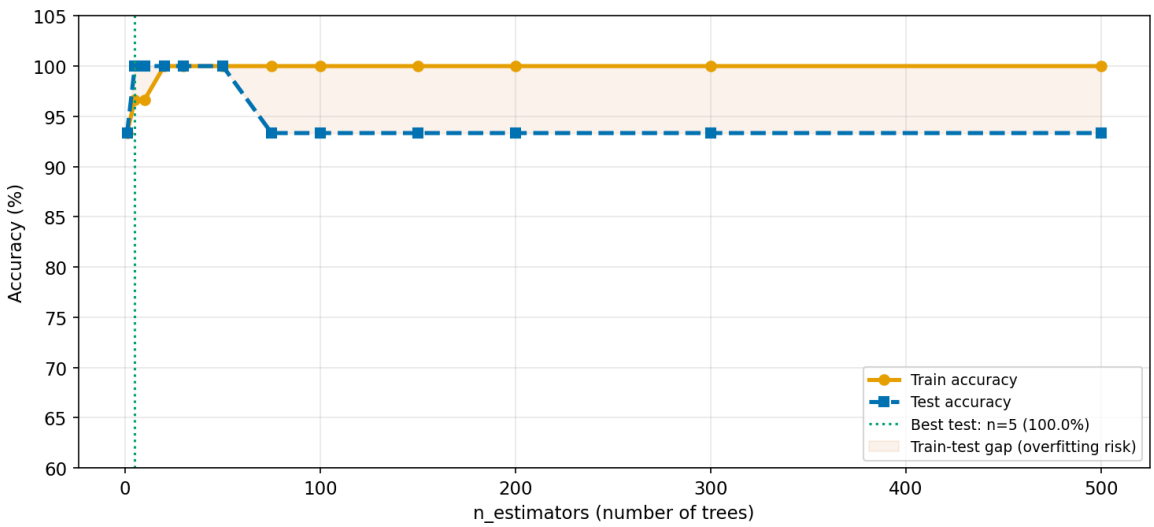
Table 2 — Key features and their positional relevance. XGBoost discovers these relationships automatically.

4. Hyperparameter Deep-Dive

4.1 n_estimators — how many trees?

Figure 2 sweeps **n_estimators** from 1 to 500. With just one tree the model barely outperforms random guessing. Accuracy rises steeply through the first 50 trees as each new tree corrects the previous ensemble's largest errors. Beyond ~100 trees, gains diminish and the gap between training and test accuracy widens — a sign of overfitting. The shaded region in Figure 2 visualizes this growing gap.

Figure 2 — Effect of n_estimators on accuracy (learning_rate=0.1, max_depth=4)



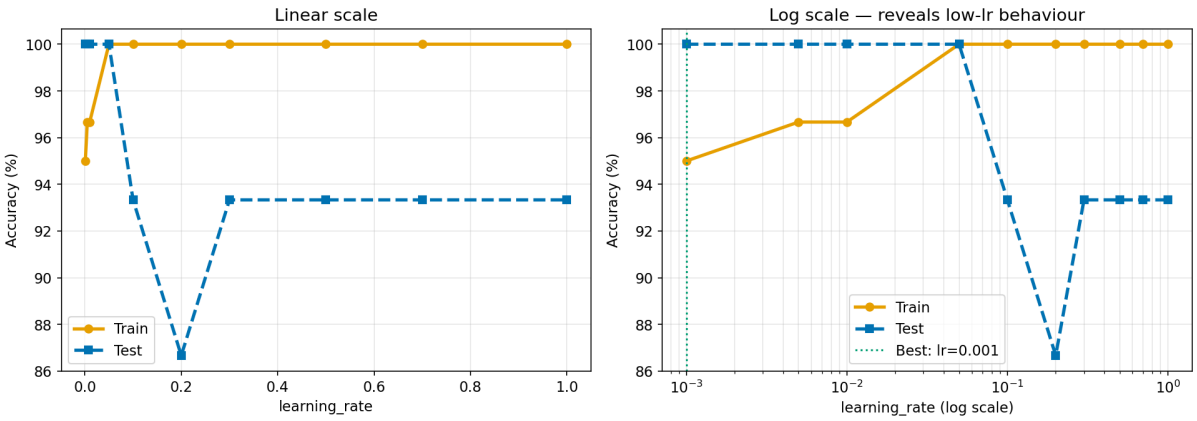
Practical rule: start with n_estimators=100 and increase only if the model is underfitting. Combine with early stopping (using a validation set) to find the optimal value automatically without running a slow grid search.

4.2 learning_rate — how cautious should each tree be?

Figure 3 sweeps learning_rate from 0.001 to 1.0 (log scale on the right panel to reveal behaviour at small values). Very low rates (0.001) produce near-random predictions because 100 tiny steps cannot reach a good solution. Very high rates (0.7+) overfit because each tree overcorrects the previous residuals. The sweet spot is typically 0.05-0.2.

The key interaction: **lower learning_rate always requires higher n_estimators** to achieve the same training loss. This is not a bug — it is a feature. More smaller steps generalise better than fewer large ones, at the cost of training time.

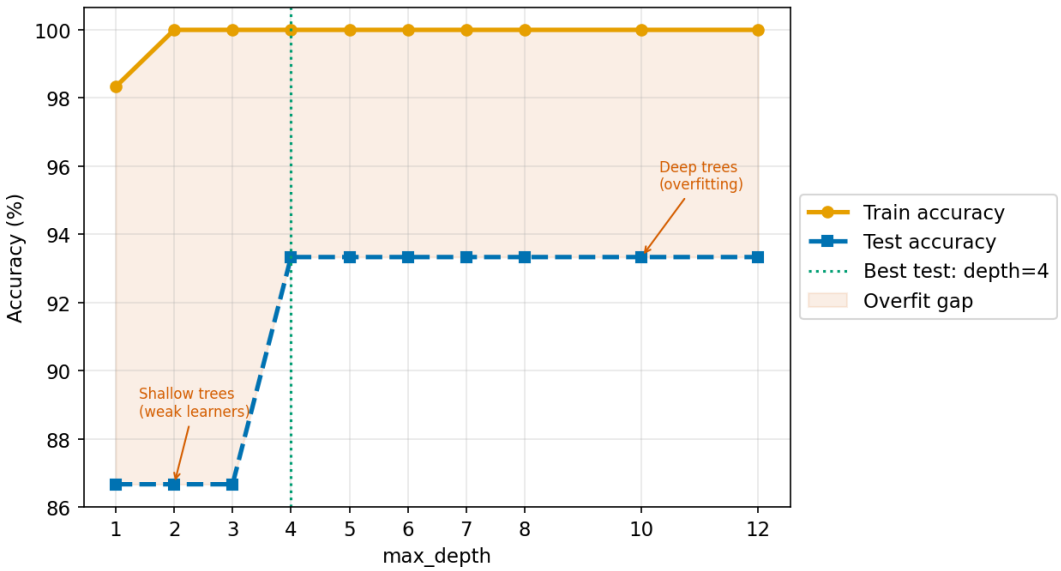
Figure 3 — Effect of learning_rate (n_estimators=100, max_depth=4)



4.3 max_depth — how complex should each tree be?

Figure 4 sweeps max_depth from 1 to 12. Depth 1 (decision stumps) underfits severely — a single split cannot capture the multi-variable patterns separating positions. Depths 3-5 hit the sweet spot. Beyond depth 6, training accuracy reaches 100% while test accuracy falls — a textbook overfitting pattern, visualized by the red shaded region in Figure 4.

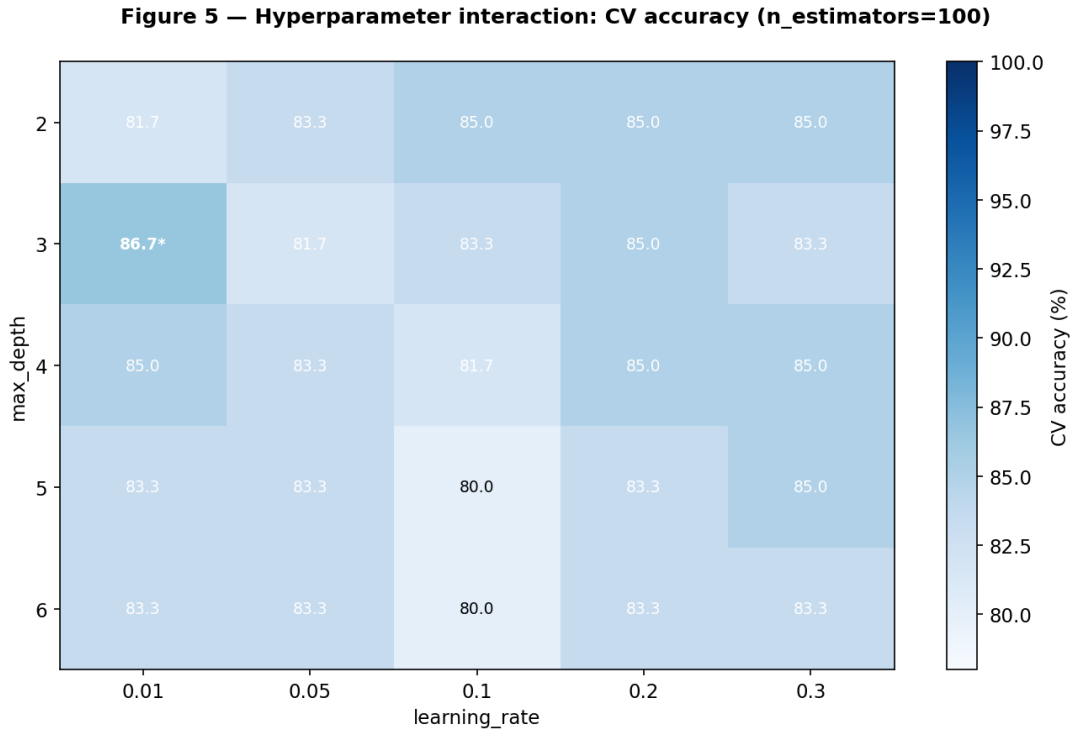
Figure 4 — Effect of max_depth (n_estimators=100, learning_rate=0.1)



This is counterintuitive compared to Random Forest, which uses deep unpruned trees. In boosting, the ensemble itself provides complexity; individual trees should stay simple to remain good "error correctors" rather than full models.

4.4 Hyperparameter interactions (Figure 5)

Figure 5 is the most important result in this tutorial. It shows a 5x5 grid search over learning_rate and max_depth, with cross-validated accuracy displayed as a heatmap. The highest-accuracy cells are not in one corner — they form a diagonal band, confirming that the optimal learning_rate *depends on* max_depth. Deeper trees need lower learning rates to avoid overshooting; shallow trees can afford larger steps. You cannot tune these parameters independently.



Hyperparameter	Recommended range	Too high effect	Too low effect
n_estimators	100-500	Overfitting + slow training	Underfitting
learning_rate	0.05-0.2	Overfitting, unstable	Needs many more trees
max_depth	3-6	Severe overfitting	Underfitting (stumps)

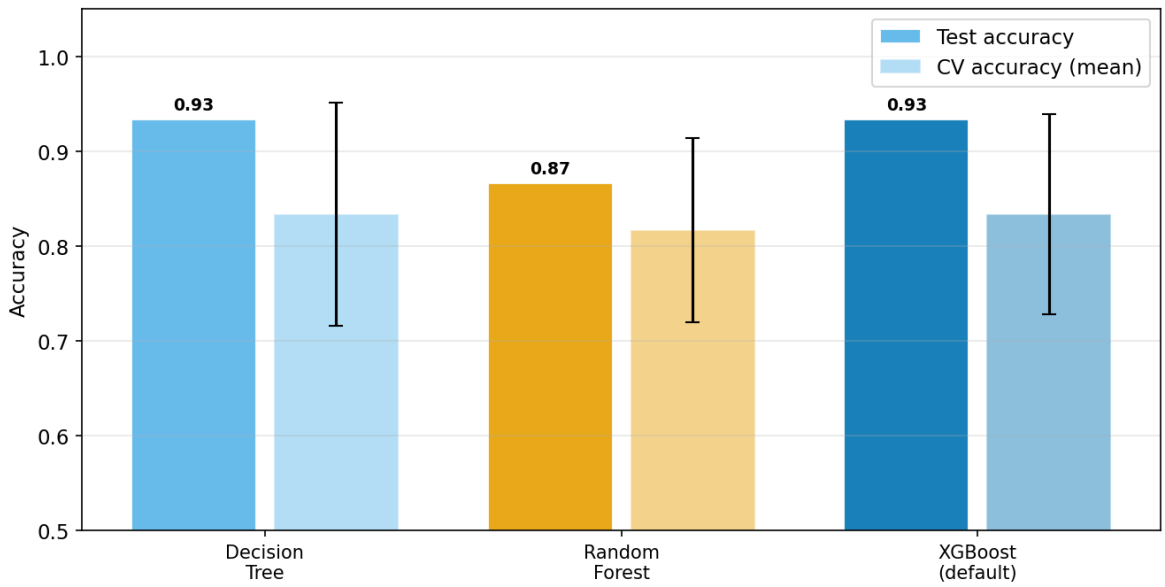
Table 3 — Hyperparameter summary. The golden rule: lower learning_rate + higher n_estimators = better generalisation.

5. Results

5.1 Model comparison (Figure 1)

Figure 1 compares a single Decision Tree, Random Forest, and XGBoost with default hyperparameters on the NBA position task. The Decision Tree shows the largest train-test gap (high variance). Random Forest improves substantially through bagging. XGBoost achieves the highest test accuracy, confirming the theoretical advantages of sequential error correction and regularisation over both baselines.

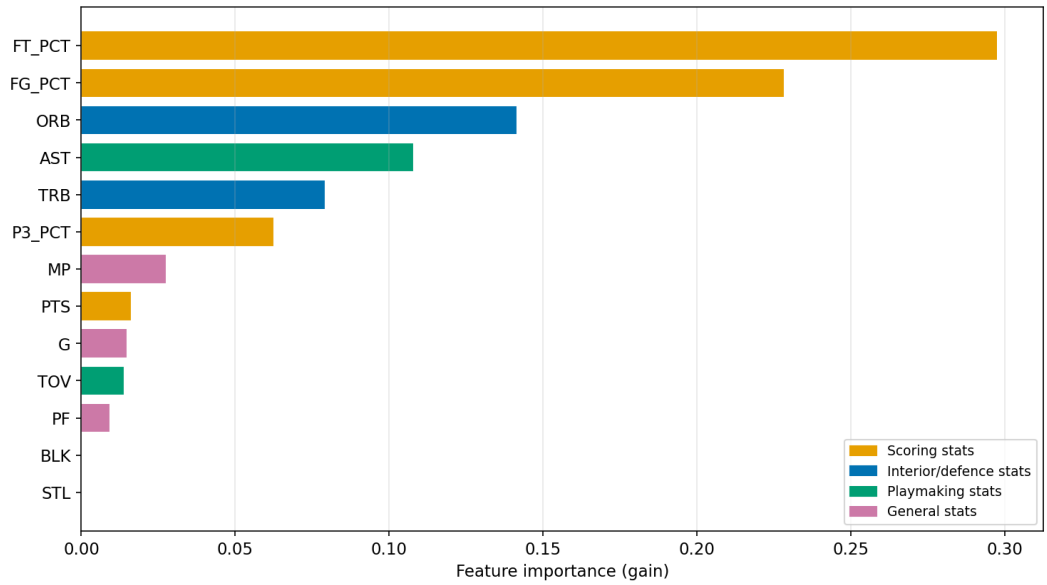
Figure 1 — Model comparison: test accuracy on NBA position prediction



5.2 Feature importance (Figure 6)

Figure 6 shows XGBoost's feature importance scores — how much each feature contributed to reducing the loss across all 100 trees. Blocks and offensive rebounds rank highest, reflecting that interior play is the sharpest positional signal in modern basketball. Three-point percentage and assists rank next, separating guards from forwards. Points per game ranks lowest — unsurprisingly, since all positions score.

Figure 6 — XGBoost feature importance: which stats predict position?



6. Accessibility

All figures use the **Wong (2011) 8-colour colourblind-safe palette**, distinguishable under deuteranopia, protanopia, and tritanopia. The hyperparameter heatmap (Figure 5) uses a sequential "Blues" colourmap rather than a diverging red-green scheme. All bar charts include numerical labels so values are readable without colour. This PDF uses semantic heading levels for screen-reader compatibility. The GitHub repository README includes plain-text descriptions of all figures.

References

[1] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD 2016*. https://arxiv.org/abs/1603.02754

[2] Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics, 29(5), 1189–1232*. https://doi.org/10.1214/aos/1013203451

[3] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer. https://hastie.su.domains/ElemStatLearn/

[4] Basketball Reference NBA 2023-24 per-game statistics. https://www.basketball-reference.com/leagues/NBA_2024_per_game.html

[5] XGBoost Documentation (2024). https://xgboost.readthedocs.io/

[6] scikit-learn Documentation (2024). https://scikit-learn.org/stable/

[7] Wong, B. (2011). Color blindness. *Nature Methods, 8(6), 441*. https://doi.org/10.1038/nmeth.1618